Appello del 16 Gennaio 2018



Politecnico di Milano

Anno accademico 2017-2018

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Mottola	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica della classe `Bunch`

```
public class Bunch<T> {
// OVERVIEW: Una collezione di oggetti di tipo T.
// Gli elementi sono disposti in ordine qualunque e sono  di molteplicità qualunque.
// Il valore null non  fa parte della collezione
// Usa equals per confrontare gli oggetti

  //@ensures (*costruisce il bunch vuoto*)
  public Bunch();

  // Aggiunge x a this
  // Lancia eccezione se x e' null
  public void ins(T x) throws NullPointerException;

  // Elimina una comparsa di x
  // Restituisce il numero di elementi rimanenti uguali a x.
  // Lancia eccezione se x e' null
  public int del(T x)  throws NullPointerException;

  // Restituisce il numero di elementi uguali a x
  public /*@ pure @*/ int num(T x)
}
```

Domanda a

Scrivere la specifica JML del metodo `del(T x)` e un invariante pubblico per la classe.

Soluzione

```
public invariant (\forall T t;; num(t) \ge 0);

//@ensures x!=null &&
//@ (\forall T t;!t.equals(x); num(t) == \old(num(t)) &&
//@ \old(num(x)==0) ? num(x) ==0 && \result==0
//@                 : num(x) ==\old(num(x)-1) && \result==num(x);
//@signals(NullPointerException e) x == null;
public int del(T x) throws NullPointerException
```

Domanda b

Completare, con AF e RI (esprimibili con opportuni inviarianti) e con il codice del metodo `del(T x)`, la seguente implementazione della classe `Bunch`. Il rep è costituito da una lista di coppie, ciascuna contenente un elemento, se presente, e la sua molteplicità. I dettagli possono essere ricavati dall'esame del codice.

```
public class Bunch<T> {
//rep:
     private List<Pair> elem;
     private static class Pair {
          // OVERVIEW: Tipo immutabile rappresentante una coppia (T,int)
          T t;
          int i;
          Pair(T y, int j) {
               t = y;
               i = j;
```

```
        }
        boolean contains(T x) {
            return (x.equals(t));
        }
    }
    public Bunch(){
        elem = new ArrayList<T>;
    }
    private int find(T x) {
        for (int j=0; j<elem.size(); j++;) {
            Pair p = elem.get(j);
            if (p.contains(x)) return p.i;
        }
        return -1;
    }
    public /*@ pure @*/ int num(T x) {
        int j = find(x);
        if (j==-1) return 0;
        return elem.get(j).i;
    }
    public void ins(T x) throws NullPointerException {
        if (x== null) throw new NullPointerException();
        i = find(x);
        if (i>-1) elem.set(i,new Pair(x,1+p.i));
        else elem.add(new Pair(x,1));
        return this;
    }
    public int del(T x) throws NullPointerException {
        if (x== null) throw new NullPointerException();
        int j = find(x);
        if (j==-1) return 0;
        ....................................
        ....................................
        ....................................
        ....................................
        ....................................
        ....................................
        ....................................
    }
}
```

Soluzione

```
    public int del(T x) throws NullPointerException {
      if (x== null) throw new NullPointerException();
      int j =find(x);
      if (j==-1) return 0;
      int temp=elem.get(j).i-1;
      if (temp==0) {
        elem.remove(j);
        return 0;
      }
      elem.get(j).i=temp;
      return temp;
    }
```

RI:

Soluzione

```
private invariant elem != null &&
(\forall int j; 0<=j && j<elem.size();
 elem.get(j).i>0);
```

AF:

Soluzione

```
private invariant
(\forall int j; 0<=j && j<elem.size();
    num(elem.get(j).t)== elem.get(j).i) &&
(\forall T x;;
    !(\exists int j; 0<=j && j<elem.size(); x.equals(elem.get(j).t)
    ==> num(x)==0;
```

Esercizio 2

Si consideri il seguente scheletro di classe Java:

```
public class DoubleBuffer {
    private String val1, val2;

    public DoubleBuffer() { val1 = val2 = null; }

    // Sospende il chiamante se val1==null e rimuove il valore prima di ritornarlo
    public String get1() { ... }

    // Sospende il chiamante se val1!=null
    public void put1(String v) { ... }

    // Sospende il chiamante se val2==null e rimuove il valore prima di ritornarlo
    public String get2() { ... }

    // Sospende il chiamante se val1!=null
    public void put2(String v) { ... }
}
```

Domanda a

Si implementino i metodi `get1`, `put1`, `get2` e `put2`. Si massimizzi il parallelismo tra i thread chiamanti.

Soluzione

```
public class DoubleBuffer {
    private String val1, val2;
    private Object lock1, lock2;
    public DoubleBuffer() {
        val1 = val2 = null;
        lock1 = new Object(); lock2 = new Object();
    }
    public String get1() throws InterruptedException {
        String v=null;
        synchronized(lock1) {
            while(val1==null) lock1.wait();
            v = val1;
            val1=null;
            lock1.notifyAll();
        }
        return v;
    }
    public void put1(String v) throws InterruptedException {
        synchronized(lock1) {
            while(val1!=null) lock1.wait();
            val1 = v;
            lock1.notifyAll();
        }
    }
    public String get2() throws InterruptedException {
        String v=null;
        synchronized(lock2) {
            while(val2==null) lock2.wait();
            v = val2;
            val2=null;
            lock2.notifyAll();
        }
```

```
        return v;
    }
    public void put2(String v) throws InterruptedException {
        synchronized(lock2) {
            while(val2!=null) lock2.wait();
            val2 = v;
            lock2.notifyAll();
        }
    }
}
```

Domanda b

Si aggiunga alla classe precedente il metodo `String readLongest()` che sospende il chiamante se `val1` oppure `val2` sono nulli per poi ritornare il più lungo dei due valori (si usi `String.length()`).

Soluzione

```
public String readLongest() throws InterruptedException {
    synchronized(lock1) {
        while(val1!=null) lock1.wait();
        synchronized(lock2) {
            while(val2!=null) lock2.wait();
            if(val1.length()>val2.length()) return val1;
            else return val2;
        }
    }
}
```

Esercizio 3

Si consideri la classe `Couple` così definita:

```
public class Couple {
    public int val1;
    public int val2;
}
```

e una lista `data` di istanze di tale classe, opportunamente inizializzata:

```
List<Couple> data = new ArrayList<Couple>();
data.add(...); ...; data.add(...);
```

Si scriva un frammento di programma Java 8 che usa lo stile funzionale per restituire la somma di tutti i secondi elementi (`val2`) delle coppie nella lista `data` il cui primo elemento sia maggiore o uguale a zero.

Soluzione

```
int result = data.stream().filter(v -> v.val1>=0)
                  .map(v -> v.val2)
                  .reduce(0, (tot, val) -> tot+val);
```

Esercizio 4

Si consideri il seguente codice. Operando su un array di esattamente tre elementi, il codice deve riorganizzarli in ordine decrescente.

```
public  int[] order(int v[]) {
  int tmp;
    if (v[0]<v[1]) {
      tmp = v[0];
      v[1] = v[1];
      v[1] = tmp;
    }
    if (v[1]<v[2]) {
      tmp = v[1];
      v[1] = v[2];
      v[2] = tmp;
    }
    return v;
}
```

Si identifichi il numero minimo di casi di test in maniera da soddisfare (separatamente) ciascuno dei seguenti criteri e insieme trovare tutti gli eventuali errori presenti nel codice.

1. copertura di tutte le istruzioni;
2. copertura di tutte le diramazioni;
3. copertura di tutte le condizioni.

Giustificate le vostre risposte.

Soluzione

Ci sono evidentemente due errori nel codice. Nel ramo `true` del primo if, i due valori non vengono scambiati correttamente. Invece, `v[0]` viene duplicato in `v[1]` che scompare. Inoltre, il codice non copre ad esempio il caso in cui l'array in ingresso sia completamente ordinato in maniera crescente. Il terzo elemento non ha maniera di diventare il primo.

Per coprire tutte le istruzioni (domanda 1) e insieme trovare entrambi gli errori, é sufficiente un singolo caso di test del tipo `[2,3,4]`. Questo produce in output `[2,4,2]`. Il numero 3 è scomparso, l'array non è ordinato in maniera decrescente, ma tutte le istruzioni sono coperte.

Per coprire tutte le diramazioni (domanda 2) e insieme trovare entrambi gli errori, partiamo ancora da `[2,3,4]` che copre le diramazioni `true` dei due `if` e trova tutti gli errori. Manca un caso di test che copra le diramazioni `false` dei due `if`. Per questo basta aggiungere un caso di test con un array già ordinato in maniera decrescente, ad esempio `[2,3,4]`.

Lo stesso insieme di casi di test è ovviamente sufficiente a coprire tutte le condizioni (domanda 3).